



CAB420 ASSESSMENT 1B

REPORT

RORY BLAIR, N10773398



MAY 11, 2025

CAB420
MACHINE LEARNING

PROBLEM 1:

Pre-Processing:

Processing data is a critical part of machine learning, especially regarding training models used for classification, such as identifying people or objects. The training data is collected via a pre-defined function called 'load_data', which is provided in the template.

The images from the output of this function are then resized from their original 128x64 resolution to 64x32. The resizing of the data is also completed by a pre-defined function called 'resize'. The data is resized to half its original size to allow for faster computational speed, but this has the disadvantage of less accurate analysis as the pictures would be compressed, allowing fewer pixels to be analysed.

No colour augmentations are applied to the data, as preserving true colours is essential to avoid misidentification. Making colour augmentations to photos could allow for colours to blend in with other colours. An example of why no colour augmentation is undertaken can be seen in Figure 1.1 [1], which shows how certain colours appear the same in different colour filters.



Figure 1.1: Colour Filter augmentations

Selection of Model Approaches:

Two approaches are being tested and analysed to allow one of the following models to classify the same person using multiple photos of one person within different stances and environments. The first approach that is being assessed is a non-Deep Learning (DL) approach, whilst the second approach is a DL approach. This is essentially testing a faster model against a slower model, with both having multiple advantages and disadvantages when it comes to classification.

The non-DL approach that is being tested is a mixture of Principal Component Analysis (PCA) and Linear Discriminant Analysis (LDA). PCA is essential in this approach as it allows for image compression, which shrinks the data's dimensions while allowing for important information to stay intact. PCA can also be used to remove redundant data and find hidden patterns within the data. [2]. LDA is necessary in this approach it allows data to be segregated into multiple different classes, each class having its unique patterns. [3] This approach first utilises the PCA to allow for further processing, whilst LDA uses the data processed from the PCA to separate the data into 250 different classes. This approach uses examples from the week 7 practical and uses a trial and error method to test for the best hyperparameter values, such as 'n_components' and 'top1_acc'. [4]

The DL approach that is being assessed employs a Triplet network with a VGG-like CNN backbone, which uses hard negative mining and triplet loss. The CNN has three convolutional blocks that consist of 24, 48, and 96, followed by one batch normalisation layer and an embedding layer. This approach utilises the example used in the week 8 practical with some modification. [5] Unlike the example, this approach has better convolutional blocks, allowing for more thorough analysis of the data within each layer. However, the layers are low enough to allow for a balance between computational power and speed. Triplet Loss is implemented to help the model with classification using three groups called anchor item(anchor), similar item(positive), and dissimilar item(negative). The objective of triplet loss is to support the model by telling the model that the anchor should be closer to a similar item than a dissimilar item. [6] Hard Negative mining is implemented to support the triplet loss section by selecting what the model considers hard examples. This negative mining allows for the model to select harder examples for the triplet loss to make improvements on how it analyzes the data. This model was trained using a batch size of 64 and an epoch of 50, allowing for the model to truly learn a lot.

Compare 2 Models:

Both approaches use the same dataset, and both sort their pictures into sets before training the models. To analyse the performance of the models, a section of the template given was modified to allow CMC curves from both models to be plotted on the same graph. The 'ranked hist', Top 1, Top 5, Top 10, Top 50, Top 100, Top 150, and the CMC curve are also computed to allow each model to be analysed at a particular point within the training. These values are displayed in Table 1.1, which allows for the models to be easier to analyse.

	Non-DL (PCA+LDA)	DL (Triplet Network)	Variance
TOP 1	0.17	0.29	0.12
TOP 5	0.30	0.52	0.22
TOP 10	0.38	0.66	0.28
TOP 50	0.66	0.92	0.26
TOP 100	0.81	0.97	0.16
TOP 150	0.90	0.99	0.09

Table 1.1: Top numbers comparison

As seen in Table 1.1, both models start out having a low Top-1 accuracy, with the non-DL model starting at 0.17 accuracy and the DL method starting at 0.29 accuracy. This gap between model accuracies widens significantly by Top-5 as the DL method now has an accuracy of 0.52, a massive jump from 0.29. This trend continues with the variance between model accuracies growing until around Top-10 or Top-50, before the models' accuracies start to converge to where the variance at Top-150 is 0.09.

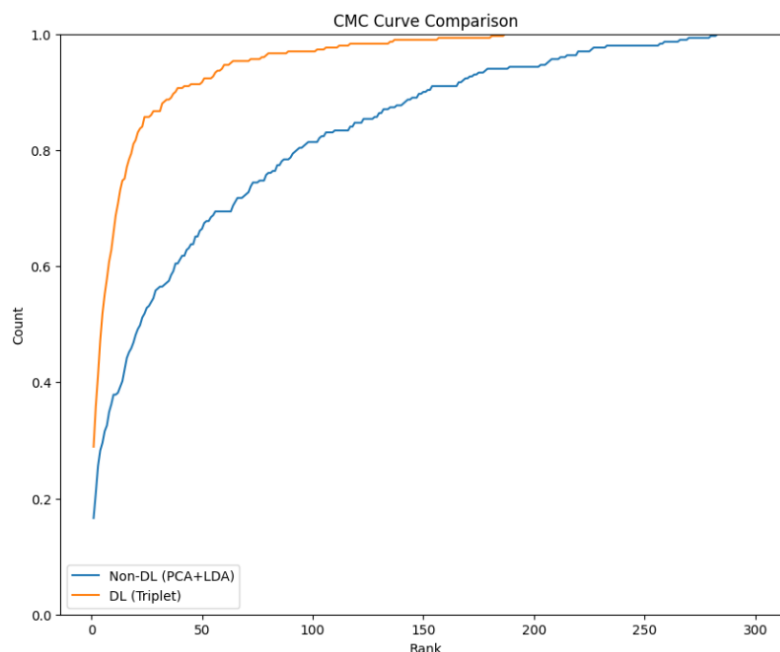


Figure 1.2: CMC Curve Comparison

These models' accuracies can also be visually displayed in Figure 1.2, where it is seen that after Top-1, the DL model quickly shoots up, reflecting what the table values suggest happened. The DL model quickly learns because of the triplet loss and hard negative mining, allowing it to identify and correct hard examples. This can also be seen in Figures 1.3 and 1.4, which show that the non-DL has double-digit ranks for only the 1st three ranks before dropping to exceptionally low numbers around rank 20, whilst the DL has double-digit ranks for a lot longer but still drops to very low numbers around rank 20. Although the DL model captures the hard examples, it also seems to have a downfall when it comes to some of the simple cases as well.

```
ranked hist [50. 13. 14. 8. 4. 6. 3. 7. 4. 5. 0. 1. 3. 3. 6. 6. 3. 2.
 3. 4. 3. 2. 4. 2. 3. 1. 2. 2. 4. 1. 1. 0. 1. 1. 1. 3.
 2. 4. 0. 2. 2. 0. 3. 1. 2. 0. 4. 0. 3. 1. 3. 1. 0. 2.
 1. 2. 0. 0. 0. 0. 0. 0. 0. 3. 2. 2. 0. 0. 1. 1. 1. 3.
 2. 0. 0. 1. 0. 0. 3. 1. 0. 1. 0. 3. 0. 2. 1. 0. 0. 1.
 2. 1. 1. 1. 0. 1. 1. 1. 0. 0. 0. 2. 1. 0. 2. 0. 0.
 0. 1. 0. 0. 0. 0. 0. 0. 2. 0. 2. 0. 0. 2. 0. 0. 0.
 0. 1. 0. 2. 0. 2. 0. 0. 1. 0. 0. 1. 0. 0. 1. 1. 1. 0.
 1. 0. 0. 2. 0. 1. 0. 1. 0. 2. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 2. 0. 1. 1. 0. 1. 0. 1. 0. 1. 0. 0. 1. 1. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 1. 0. 0. 1. 1. 1. 0. 0. 0. 1. 0. 0. 1. 0.
 0. 0. 0. 2. 0. 0. 0. 0. 0. 1. 1. 0. 0. 0. 0. 0. 1. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 1. 0. 1. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 1.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 1. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
```

Figure 1.3: Non-DL Ranked Hist

```
ranked hist [87. 21. 16. 18. 14. 10. 8. 9. 6. 9. 9. 6. 7. 5. 1. 6. 4. 3.
 5. 2. 4. 2. 1. 5. 0. 0. 1. 2. 0. 0. 0. 4. 1. 1. 0. 1.
 2. 1. 2. 0. 0. 1. 0. 0. 1. 0. 0. 0. 0. 1. 2. 0. 0. 0.
 1. 2. 1. 0. 1. 2. 0. 0. 1. 1. 0. 0. 0. 0. 0. 0. 0. 1.
 0. 0. 0. 0. 1. 0. 1. 1. 0. 0. 0. 0. 0. 0. 0. 1. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 1. 0. 0.
 0. 0. 0. 1. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 1. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0.
 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
```

Figure 1.4: DL Ranked Hist

The analysis of these two models also shows their apparent advantages and disadvantages. The main advantage of the non-DL model is the speed at which it trains, usually finishing training under 1 minute. Although the non-DL model has the disadvantage of not being able to detect certain lighting and poses within an image, as it relies on raw pixel values. The main advantage of the DL method is that, via the use of triplet loss and hard negative mining, the model can correctly identify the same person within tricky situations, unlike the non-DL model. However, the disadvantage of the DL model is that computational speed is sacrificed computational power for a more accurate result. This disadvantage can be balanced by setting the batch size and epoch numbers to certain values. (Code to recreate equivalent results can be seen in Appendix 3A)

Ethical Concerns:

These person re-identification systems raise crucial ethical concerns regarding the data collection, how the collected data is used, and the limitations of current re-identification systems. Some of these concerns already occur, such as the use of facial recognition in devices and worldwide surveillance, and if data collected via these methods is accessed by outside users, many risks are created. One such risk is how an outside user may attempt to identify a particular individual based on a photo used by a re-identification system.

Training a re-identification system usually includes collecting enormous amounts of data, with or without an individual's consent. One example of this problem occurring was when the Office of the Australian Information Commissioner (OAIC) found that Bunnings breached privacy laws. This privacy was breached by the use of a facial recognition system capturing data on customers without their consent [7]. Once data is collected, it is also used for a reason; in most cases, it is used to identify individuals and track them, which can lead to potential risks if misused. This concern about data usage is highlighted in the Bunnings case, where Bunnings aimed to protect staff from violence but violated customer privacy at the same time, creating a dilemma whether staff safety justifies customer privacy. Of course, there is also a limitation of the re-identification system, which can be seen in the previous section. As seen, the models created are not very accurate; thus, if a system using either model were used, serious mistakes could be made.

PROBLEM 2:

Pre-Processing:

Pre-processing was applied to the data being used in both the ‘from-scratch’ (FS) model and the ‘fine-tuned MobileNetV3Small’ (FTM) model, ensuring a fair comparison between the two models. There were three alterations made to the data, which were the sizing, colour conversion, and segmentation masks. The only pre-processing applied to the data that affects both models was the image size was reduced for a similar reason as the previous problem. The images were resized from their original 400x400 resolution to their converted 128x128, allowing for faster computational time, but not allowing more details to be analysed. The second alteration was the colour conversion, which was used for scaling RGB values so that the FTM model could compute the data. The final alteration was that the segmentation masks were converted to binary, resized, and cast to allow compatibility with the FTM model.

Two Model Details:

The FS model was very similar to the DL model in Problem 1, with certain modifications, as this model identifies dogs and cats as opposed to people. It still uses a VGG-like backbone but uses different sections after the backbone, such as a classification head and a segmentation head. The backbone still uses three convolutional blocks but has an upgrade in terms of the filters, specifically 32, 64, and 128. The backbone also includes a three-by-three kernel, batch normalisation, and max pooling, which helps the model correctly identify valuable information within certain sections of the images. [8] This FS model is inspired by the examples shown in the Week 9 practical, which balances computational time and power, so that it is not too slow or too high. [9] The output from the backbone is then used within the classification head, which applies a global average pooling, before giving the classification head output to a dense layer that includes softmax. The segmentation head is used throughout the model to help define edges within an image, allowing the model to analyse certain parts of an image further.

The FTM model was a brand-new model that did not appear in any other weekly lectures, practicals, or examples. It is also very similar to the FS model except that the backbone has changed from a new untrained model to a pre-trained model that contains ImageNet weights, which usually helps with prediction success, leading to more training to be undertaken. [10]. The classification and segmentation heads are also used again and have not changed since the FS model as to allow for proper comparison. No convolutional filter is necessary as the MobileNetV3 comes with pre-trained filters, and only needs to be enabled or disabled, depending on what is required.

Both models compile using the Adam optimiser with a learning rate of 0.001 and display the class output accuracy and the segmentation output accuracy. Both models are fitted using segmentation training with 100 epochs and a maximum training time of 15 minutes, to allow for a reasonable amount of training. 100 epochs are used so that the models use the whole 15 minutes, allowing them to be trained to their best. The main difference between the training of these two models is that the FTM model trains two parts separately, whilst the FS model trains the entire model at once. Initially, the backbone was unable to be trained, only allowing the classification and segmentation heads to be trained first, using only 10 epochs. After that training, the backbone was then trained again with the backbone enabled for the full 15 minutes. This is done as the backbone is already pre-trained to a certain extent, and the classification and segmentation heads are the new pieces of the model that need some training before testing the model

Compare 2 Models:

An evaluation was conducted on each model using 100 epochs, assessing each model's ability in classification accuracy and segmentation Intersection of Union (IoU). The classification accuracy would allow for the analysis of how often the models would correctly classify a breed. The segmentation IoU would help quantify how well an actual segmentation aligns with a predicted segmentation. [11]. These metrics, as well as confusion matrices for each model's classification and segmentation, are also produced, as seen in Table 2.1, Figures 2.1 and 2.2. In Table 2.1, it is seen that the FTM model significantly has an advantage over the FS model in terms of classification accuracy, which is most likely due to the FTM having been pre-trained. However, there isn't a major variance between the FS and FTM when it comes to Segmentation IoU, as they both roughly have the same training time when it comes to the segmentation head. The only reason the FTM model has a higher score is due to the segmentation head being trained by itself before training the whole model.

	From-Scratch	MobileNetV3	Variance
Classification Accuracy	0.25	0.62	0.37
Segmentation IoU	0.71	0.81	0.10

Table 2.1: Metric Comparison

As seen in Figure 2.1, the classification confusion matrix for the FTM model has a nearly clear diagonal line, signifying that the model is accurate when it comes to classifying certain breeds. However, the classification confusion matrix for the FS model does not indicate a diagonal line, only has great accuracy when it comes to classifying two breeds. These classification confusion matrices support what the accuracy values within Table 2.1 display.

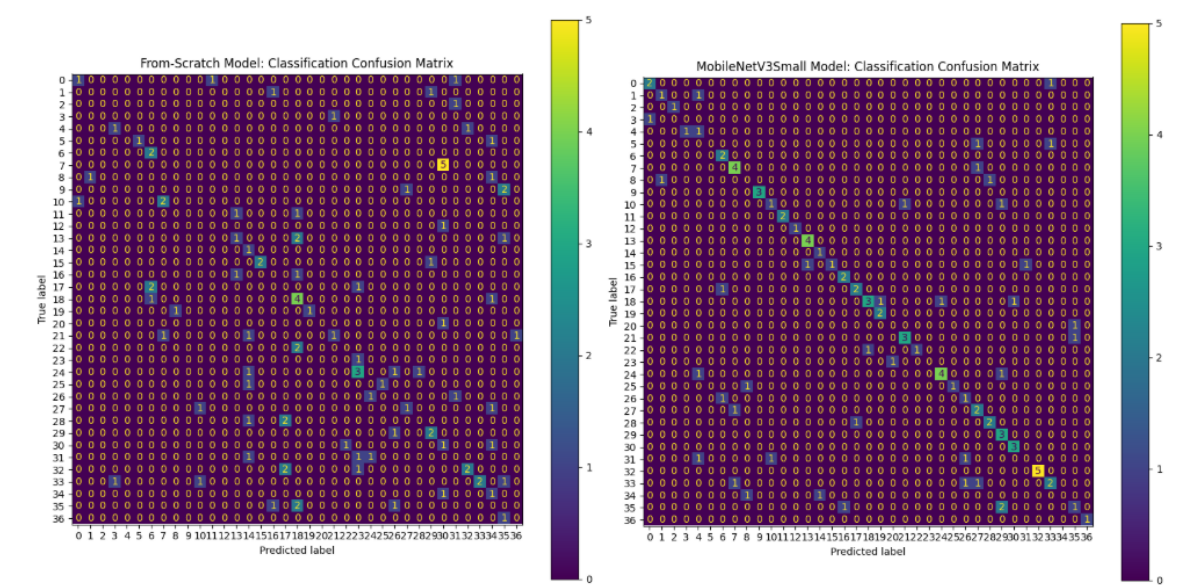


Figure 2.1: FS (LEFT) and FTM (RIGHT) Classification Confusion Matrix

As seen in Figure 2.2, both models have roughly the same problems when it comes to classifying backgrounds and foregrounds. Both models tend to classify backgrounds correctly but have a difficult time correctly classifying foregrounds, which is seen in the bottom right corner of the confusion matrices. If it classified foregrounds correctly it would be yellow, meaning it is accurate at identifying foregrounds and background equally. These segmentation confusion matrices support the what the segmentation IoU values within Tab 2.1 display.

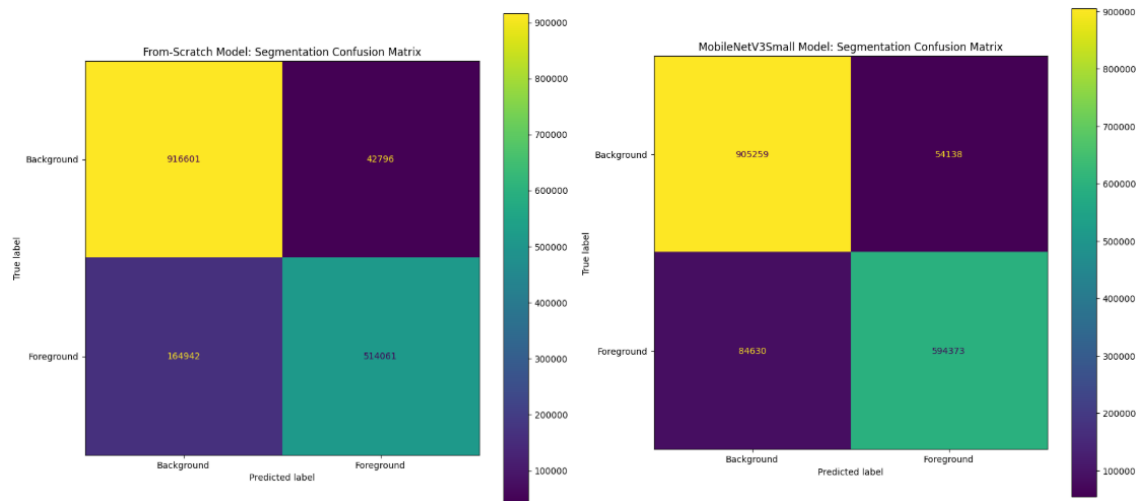


Figure 2.2: FS (LEFT) and FTM (RIGHT) Segmentation Confusion Matrix

Improvements & Mitigations Strategies 1:

There is a massive amount of mitigation made and improvements that could be made that need to be discussed. These improvements could not be made due to them having a corresponding mitigation/issue that limits the models to their basic versions.

The first improvement that could be made is to prioritise the segmentation head during training the models, as if the models could distinguish which sections to concentrate on. This improvement was not implemented as the coding environment that was used would not be able to complete the training. Google Colab on a T4 runtime was used, giving it a limit on computational power before crashing the Colab instance. Most of the improvements that could be made weren't implemented due to the Google Colab limitations.

The second improvement that could be made in the future was the image size and the batch size, both of which were decreased to allow the Colab instance to complete. Increasing the image size would allow for more accurate segmentations as there would be more pixels, thus a clear border between background and foreground. Increasing batch size would allow more beneficial training allowing each epoch to train on more examples and from it. Yet again, both of these improvements weren't incorporated due to the computational power within the Google Colab instance.

The Third improvement that could have been made was to increase the time allowed to train each model or to have the model train until the accuracy converges. Allowing the model's accuracies to converge allows the model to train until a model's accuracy settles to a certain point where the model is not getting significantly more accurate. Allowing the models to train for long should also help increase the accuracy and segmentation IoU, but as seen in Table 2.2, this can also be detrimental. As seen, the FS model has gotten significantly better with more training time, whilst the FTM model got slightly better in accuracy but got worse in the Segmentation IoU (Code to recreate equivalent results can be seen in Appendix 3B).

	From-Scratch	MobileNetV3	Variance
Classification Accuracy	0.46	0.65	0.19
Segmentation IoU	0.78	0.80	0.02
Epochs	94	99	5

Table 2.2: Metric Comparison on 30 Minute

References (IEEE):

- [1] Colour Blind Awareness, “Types of Colour Blindness,” Colour Blind Awareness, [Online]. Available: <https://www.colourblindawareness.org/colour-blindness/types-of-colour-blindness/>. [Accessed 07 05 2025].
- [2] IBM, “What is principal component analysis (PCA)?,” IBM, 08 12 2023. [Online]. Available: <https://www.ibm.com/think/topics/principal-component-analysis>. [Accessed 07 05 2025].
- [3] IBM, “What is linear discriminant analysis (LDA)?,” IBM, 27 11 2023. [Online]. Available: [https://www.ibm.com/think/topics/linear-discriminant-analysis#:~:text=Linear%20discriminant%20analysis%20\(LDA\)%20is,helps%20optimize%20machine%20learning%20models..](https://www.ibm.com/think/topics/linear-discriminant-analysis#:~:text=Linear%20discriminant%20analysis%20(LDA)%20is,helps%20optimize%20machine%20learning%20models..) [Accessed 07 05 2025].
- [4] S. Denman, “Week 7 - Practical Solutions,” QUT, 07 04 2025. [Online]. Available: https://canvas.qut.edu.au/courses/20364/pages/week-7-practical-solutions?module_item_id=1798289. [Accessed 07 05 2025].
- [5] S. Denman, “Week 8 - Practical Solutions,” QUT, 21 04 2025. [Online]. Available: https://canvas.qut.edu.au/courses/20364/pages/week-8-practical-solutions?module_item_id=1798295. [Accessed 07 05 2025].
- [6] D. Shah, “Triplet Loss: Intro, Implementation, Use Cases,” v7labs, 13 04 2023. [Online]. Available: <https://www.v7labs.com/blog/triplet-loss>. [Accessed 07 05 2025].
- [7] OAIC, “Bunnings breached Australians’ privacy with facial recognition tool,” OAIC, 19 11 2024. [Online]. Available: <https://www.oaic.gov.au/news/media-centre/bunnings-breached-australians-privacy-with-facial-recognition-tool>. [Accessed 07 05 2025].
- [8] Manav, “Everything you need to know about Convolutional Neural Networks (CNNs),” Medium, 29 10 2020. [Online]. Available: <https://medium.com/analytics-vidhya/everything-you-need-to-know-about-convolutional-neural-networks-cnns-3a82f7aa29c5>. [Accessed 09 05 2025].

- [9] S. Denman, “Week 9 - Practical Solutions,” QUT, 28 04 2025. [Online]. Available: https://canvas.qut.edu.au/courses/20364/pages/week-9-practical-solutions-2?module_item_id=1798309. [Accessed 09 05 2025].
- [10] Ismetgocer, “Weight Selection in Transfer Learning Models in Deep Learning: ImageNet Example,” Medium, 19 12 2023. [Online]. Available: <https://medium.com/academy-team/weight-selection-in-transfer-learning-models-in-deep-learning-imagenet-example-a60a898b0c65>. [Accessed 09 05 2025].
- [11] D. Shah, “Intersection over Union (IoU): Definition, Calculation, Code,” v7labs, 30 05 2023. [Online]. Available: <https://www.v7labs.com/blog/intersection-over-union-guide>. [Accessed 09 05 2025].

APPENDICES:

Appendix 3A: Code for Problem 1

```
#
# Utility functions for CAB420, Assignment 1B, Q1
# Author: Simon Denman (s.denman@qut.edu.au)
#
from google.colab import drive
drive.mount('/content/drive')
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'

import matplotlib.pyplot as plt      # for plotting
import numpy as np                   # for reshaping, array manipulation
import cv2                           # for image loading and colour conversion
import tensorflow as tf               # for bulk image resize
import glob
import random
import keras
from keras import layers
from keras import backend as K
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
import keras.ops as ops

# get the subject ID from the filename. A sample filename is
# 0001_cls1_001051_00.jpg, the first
# four characters are the subject ID
# fn:          the filename to parse
#
# returns: first four characters of the filename converted to an int
#
def get_subject_id_from_filename(fn):
    return int(fn[0:4])

# load the images in a directory
# base_path: path to the data
#
# returns: numpy arrays of size (samples, width, height, channels), and
# size (samples) for
#          images and thier labels
def load_directory(base_path):

    # find all images in the directory
    files = glob.glob(os.path.join(base_path, '*.jpg'))
    x = []
    y = []

    # loop through the images, loading them and extracting the subject ID
    for f in files:
        x.append(cv2.cvtColor(cv2.imread(f), cv2.COLOR_BGR2RGB) / 255.0)
        y.append(get_subject_id_from_filename(os.path.basename(f)))

    return np.array(x), np.array(y)

# Split the training set into train and validation, with a disjoint set of
# IDs in each
# X:          X data to split
```

```
# Y:          Y data to split, corresponds to X
# id_cutoff: Any IDs equal or smaller than this will be train, the rest
# will be in validation
#
# returns:    numpy arrays for the training and validation sets, which
# contain mutually exclusive sets
#             of subjects
#
def split_validation(X, Y, id_cutoff = 250):

    train_X = X[Y <= id_cutoff, :]
    train_Y = Y[Y <= id_cutoff]

    val_X = X[Y > id_cutoff, :]
    val_Y = Y[Y > id_cutoff]

    return train_X, train_Y, val_X, val_Y

# Extract gallery and probe sets from a given chunk of data. Gallery and
# probe will contain one sample each
# for each subject, with only the first and second images for each subject
# used, and the rest discarded.
# X:          X data to split
# Y:          Y data to split, corresponds to X
#
# returns:    numpy arrays containing gallery and probe sets corresponding
# to the input data. Gallery and
#             probe will both contain one image of each subject. Any subjects
# with fewer than two images
#             will be discarded
#
def sample_gallery_and_probe(X, Y):

    # storage
    gallery_X = []
    gallery_Y = []
    probe_X = []
    probe_Y = []

    # find unique IDs
    unique_ids = np.unique(Y)
    # loop through unique IDs
    for i in unique_ids:
        # pull out samples for the current ID
        samples = X[Y == i, :]
        # need at least two samples for use the ID
        if samples.shape[0] >= 2:
            # first sample for a subject stored in the gallery
            gallery_X.append(samples[0, :])
            gallery_Y.append(i)
            # second sample stored in the probe
            probe_X.append(samples[1, :])
            probe_Y.append(i)
            # any other images for the subject are ignored

    # return extracted data
    return np.array(gallery_X), np.array(gallery_Y), np.array(probe_X),
    np.array(probe_Y)

# load the data
```

```
# base_path: path to the data, within the directory that this points to
# there should be a 'Training'
# and 'Testing' directory
#
# returns: loaded data
#
def load_data(base_path):

    # load training data
    train_X, train_Y = load_directory(os.path.join(base_path, 'Training'))
    # split validation data
    train_X, train_Y, val_X, val_Y = split_validation(train_X, train_Y)

    # load gallery data from the test set
    gallery_X, gallery_Y = load_directory(os.path.join(base_path, 'Testing',
    'Gallery'))

    # load probe data from the test set
    probe_X, probe_Y = load_directory(os.path.join(base_path, 'Testing',
    'Probe'))

    return train_X, train_Y, val_X, val_Y, gallery_X, gallery_Y, probe_X,
    probe_Y

# Plot some images and their labels. Will plot the first 50 samples in a
# 10x5 grid
# x: array of images, of shape (samples, width, height, channels)
# y: labels of the images
#
def plot_images(x, y):
    fig = plt.figure(figsize=[15, 18])
    for i in range(50):
        ax = fig.add_subplot(5, 10, i + 1)
        ax.imshow(x[i,:], cmap=plt.get_cmap('Greys'))
        ax.set_title(y[i])
        ax.axis('off')

# vectorise an array of images, such that the shape is changed from {samples,
# width, height, channels} to
# (samples, width * height * channels)
# images: array of images to vectorise
#
# returns: vectorised array of images
#
def vectorise(images):
    # use numpy's reshape to vectorise the data
    return np.reshape(images, [len(images), -1])

# Resize an array of images
# images: array of images, of shape (samples, width, height, channels)
# new_size: tuple of the new size, (new_width, new_height)
#
# returns: resized array of images, (samples, new_width, new_height,
# channels)
#
def resize(images, new_size):
    # tensorflow has an image resize funtion that can do this in bulk
    # note the conversion back to numpy after the resize
    return tf.image.resize(images, new_size).numpy()

# Convert images to grayscale
```

```
# images: array of colour images to convert, of size (samples, width,
height, 3)
#
# returns: array of converted images, of size (samples, width, height, 1)
#
def convert_to_grayscale(images):
    # storage for converted images
    gray = []
    # loop through images
    for i in range(len(images)):
        # convert each image using openCV
        gray.append(cv2.cvtColor(images[i,:], cv2.COLOR_RGB2GRAY))
    # pack converted list as an array and return
    return np.expand_dims(np.array(gray), axis = -1)

# Create a batch of siamese data. Pairs will be evenly balanced, such that
there is an
# equal number of positive and negative pairs
# imgs: images to use to generate data, of shape (samples, width,
height, channels)
# labels: labels for the data, of shape (samples)
# batch_size: number of pairs to generate
#
# returns: image pairs and labels to indicate if the pairs are the same,
or different
#
def get_siamese_data(imgs, labels, batch_size):

    image_a = np.zeros((batch_size, np.shape(imgs)[1], np.shape(imgs)[2],
np.shape(imgs)[3]));
    image_b = np.zeros((batch_size, np.shape(imgs)[1], np.shape(imgs)[2],
np.shape(imgs)[3]));
    label = np.zeros(batch_size);

    for i in range(batch_size):

        if (i % 2 == 0):
            idx1 = random.randint(0, len(imgs) - 1)
            idx2 = random.randint(0, len(imgs) - 1)
            l = 1
            while (labels[idx1] != labels[idx2]):
                idx2 = random.randint(0, len(imgs) - 1)

        else:
            idx1 = random.randint(0, len(imgs) - 1)
            idx2 = random.randint(0, len(imgs) - 1)
            l = 0
            while (labels[idx1] == labels[idx2]):
                idx2 = random.randint(0, len(imgs) - 1)

        image_a[i, :, :, :] = imgs[idx1,:,:,:]
        image_b[i, :, :, :] = imgs[idx2,:,:,:]
        label[i] = l

    return [image_a, image_b], label

# Plot the first 10 pairs of a batch, good sanity check for pair generation
# x: images in the pairs
# y: labels of the pairs
#
def plot_pairs(x, y):
```

```

fig = plt.figure(figsize=[25, 6])
for i in range(10):
    ax = fig.add_subplot(2, 10, i*2 + 1)
    ax.imshow(x[0][i,:], cmap=plt.get_cmap('Greys'))
    ax.set_title('Pair ' + str(i) + '; Label: ' + str(y[i]))

    ax = fig.add_subplot(2, 10, i*2 + 2)
    ax.imshow(x[1][i,:], cmap=plt.get_cmap('Greys'))
    ax.set_title('Pair ' + str(i) + '; Label: ' + str(y[i]))

# Create a batch of triplet data.
#   imgs:          images to use to generate data, of shape (samples, width,
height, channels)
#   labels:        labels for the data, of shape (samples)
#   batch_size:    number of triplets to generate
#
#   returns:       triplet of the requested batch size
#
def get_triplet_data(imgs, labels, batch_size):

    image_a = np.zeros((batch_size, np.shape(imgs)[1], np.shape(imgs)[2],
np.shape(imgs)[3]));
    image_b = np.zeros((batch_size, np.shape(imgs)[1], np.shape(imgs)[2],
np.shape(imgs)[3]));
    image_c = np.zeros((batch_size, np.shape(imgs)[1], np.shape(imgs)[2],
np.shape(imgs)[3]));

    for i in range(batch_size):

        idx1 = random.randint(0, len(imgs) - 1)
        idx2 = random.randint(0, len(imgs) - 1)
        idx3 = random.randint(0, len(imgs) - 1)

        while (labels[idx1] != labels[idx2]):
            idx2 = random.randint(0, len(imgs) - 1)

        while (labels[idx1] == labels[idx3]):
            idx3 = random.randint(0, len(imgs) - 1)

        image_a[i, :, :, :] = imgs[idx1,:,:,:]
        image_b[i, :, :, :] = imgs[idx2,:,:,:]
        image_c[i, :, :, :] = imgs[idx3,:,:,:]

    return [image_a, image_b, image_c]

# Plot the first 9 triplets of a batch, good sanity check for triplet
generation
#   x: images in the triplets
#
def plot_triplets(x):
    fig = plt.figure(figsize=[25, 10])
    for i in range(9):
        ax = fig.add_subplot(3, 9, i*3 + 1)
        ax.imshow(x[0][i,:], cmap=plt.get_cmap('Greys'))
        ax.set_title('Triple ' + str(i) + ': Anchor')

        ax = fig.add_subplot(3, 9, i*3 + 2)
        ax.imshow(x[1][i,:], cmap=plt.get_cmap('Greys'))
        ax.set_title('Triple ' + str(i) + ': Positive')

        ax = fig.add_subplot(3, 9, i*3 + 3)

```

```

    ax.imshow(x[2][i,:], cmap=plt.get_cmap('Greys'))
    ax.set_title('Triple ' + str(i) + ': Negative')

# Compute a ranked histogram, which can be used to generate a CMC curve.
# This function will loop
# through all probe samples. For each probe sample it will:
# - Compare the sample to all gallery samples to get a distance between the
#   probe sample and
#   all gallery samples. In this case it is the L2 distance
# - Sort the gallery samples by how close they are to the probe samples
# - Find the location of the true match
# - Update a ranked histogram based on this
# The ranked histogram will show how many samples are matched at each rank.
# For example,
# ranked_histogram[0] will record how many samples are matched at Rank-1.
#
# This implementation assumes that there is only one sample per ID in the
# gallery set
#
# NOTE: L2 distance, used here, may not be appropriate for all problems.
# Consider the nature
# of your problem and what distance measure you should use
#
# gallery_feat: features for the gallery data, of shape (gallery_samples,
# num_features)
# gallery_Y:     IDs of the gallery samples, of shape (gallery_samples,)
# probe_feat:    features for the probe data, of shape (probe_samples,
# num_features)
# probe_Y:       IDs of the probe samples, of shape (probe_samples,)
# verbose:       bool to indicate if debug information should be printed.
# Be careful using this with
#                 large feature sets, and/or lots of samples
#
# returns:       ranked histogram matching the probe samples to the gallery
#
def get_ranked_histogram_l2_distance(gallery_feat, gallery_Y, probe_feat,
probe_Y, verbose = False):

    # storage for ranked histogram
    # length equal to number of unique subjects in the gallery
    ranked_histogram = np.zeros(len(np.unique(gallery_Y)))

    # loop over all samples in the probe set
    for i in range(len(probe_Y)):
        # get the true ID of this sample
        true_ID = probe_Y[i]
        if verbose:
            print('Searching for ID %d' % (true_ID))

        # get the distance between the current probe and the whole gallery,
        # L2 distance here. Note that L2
        # may not always be the best choice, so consider your distance metric
        # given your problem
        dist = np.linalg.norm(gallery_feat - probe_feat[i,:], axis=1, ord=2)
        if verbose:
            print(dist)

        # get the sorted order of the distances
        a = np.argsort(dist)
        # apply the order to the gallery IDs, such that the first ID in the
        # list is the closest, the second

```

```

    # ID is the second closest, and so on
    ranked = gallery_Y[a]
    if verbose:
        print('Ranked IDs for query:')
        print(a)

    # find the location of the True Match in the ranked list
    ranked_result = np.where(ranked == true_ID)[0][0]
    if verbose:
        print(ranked_result)

    # store the ranking result in the histogram
    ranked_histogram[ranked_result] += 1
    if verbose:
        print('')

    if verbose:
        print(ranked_histogram)

    return ranked_histogram

# Convert a ranked histogram to a CMC. This simply involves computing the
# cumulative sum over the histogram
# and dividing it by the length of the histogram
#   ranked_hist: ranked histogram to convert to a CMC
#
#   returns:      CMC curve
#
def ranked_hist_to_CMC(ranked_hist):

    cmc = np.zeros(len(ranked_hist))
    for i in range(len(ranked_hist)):
        cmc[i] = np.sum(ranked_hist[:i + 1])

    return (cmc / len(ranked_hist))

# plot a CMC
#   cmc: cmc data to plot
#
def plot_cmc(cmc):
    fig = plt.figure(figsize=[10, 8])
    ax = fig.add_subplot(1, 1, 1)
    ax.plot(range(1, len(cmc)+1), cmc)
    ax.set_xlabel('Rank')
    ax.set_ylabel('Count')
    ax.set_ylim([0, 1.0])
    ax.set_title('CMC Curve')

# Function to evaluate PCA + LDA model on a gallery and probe set (Non-DL)
def evaluate_pca_lda(pca_n_components, train_X_vec, train_Y, gallery_X_vec,
gallery_Y, probe_X_vec, probe_Y):
    pca = PCA(n_components=pca_n_components)
    pca.fit(train_X_vec)
    train_pca = pca.transform(train_X_vec)
    gallery_pca = pca.transform(gallery_X_vec)
    probe_pca = pca.transform(probe_X_vec)

    lda = LinearDiscriminantAnalysis()
    lda.fit(train_pca, train_Y)
    gallery_lda = lda.transform(gallery_pca)
    probe_lda = lda.transform(probe_pca)

```

```

    ranked_hist = get_ranked_histogram_l2_distance(gallery_lda, gallery_Y,
    probe_lda, probe_Y)
    cmc = ranked_hist_to_CMC(ranked_hist)
    return cmc[0] # Return Top-1 accuracy

# Convolutional block for DL model (from Week 8) with light L2 regularization
def conv_block(inputs, filters, spatial_dropout=0.0, max_pool=True):
    x = layers.Conv2D(filters=filters, kernel_size=(3,3), padding='same',
    activation='relu',
                                kernel_regularizer=keras.regularizers.l2(0.001))(inputs)
    x = layers.Conv2D(filters=filters, kernel_size=(3,3), padding='same',
    activation=None,
                                kernel_regularizer=keras.regularizers.l2(0.001))(x)
    x = layers.BatchNormalization()(x)
    x = layers.Activation('relu')(x)
    if spatial_dropout > 0.0:
        x = layers.SpatialDropout2D(spatial_dropout)(x)
    if max_pool:
        x = layers.MaxPool2D(pool_size=(2, 2))(x)
    return x

# Fully connected block for DL model (from Week 8) with light L2
regularization
def fc_block(inputs, size, dropout):
    x = layers.Dense(size, activation=None,
    kernel_regularizer=keras.regularizers.l2(0.001))(inputs)
    x = layers.BatchNormalization()(x)
    x = layers.Activation('relu')(x)
    if dropout > 0.0:
        x = layers.Dropout(dropout)(x)
    return x

# VGG-like network for DL model with adjusted filters
def vgg_net(inputs, filters, fc, spatial_dropout=0.0, dropout=0.0):
    x = inputs
    for idx, i in enumerate(filters):
        x = conv_block(x, i, spatial_dropout, not (idx == len(filters) - 1))
    x = layers.Flatten()(x)
    for i in fc:
        x = fc_block(x, i, dropout)
    return x

# Normalize vector function for triplet loss
def normalise_vector(vect):
    mag = ops.sqrt(ops.sum(ops.square(vect), axis=1))
    mag = ops.reshape(ops.tile(mag, (vect.shape[1],)), (-1, vect.shape[1]))
    return ops.divide(vect, mag)

# Triplet Loss Layer for DL model
class TripletLossLayer(layers.Layer):
    def __init__(self, alpha, **kwargs):
        self.alpha = alpha
        super(TripletLossLayer, self).__init__(**kwargs)

    def triplet_loss(self, inputs):
        anchor, positive, negative = inputs
        anchor = normalise_vector(anchor)
        positive = normalise_vector(positive)
        negative = normalise_vector(negative)

```

```

        p_dist = ops.sum(ops.square(anchor - positive), axis=-1)
        n_dist = ops.sum(ops.square(anchor - negative), axis=-1)
        return ops.sum(ops.maximum(p_dist - n_dist + self.alpha, 0), axis=0)

    def call(self, inputs):
        loss = self.triplet_loss(inputs)
        self.add_loss(loss)
        return loss

    def compute_output_shape(self, input_shape):
        return (None,)

# Hard negative mining with adjusted distance range for DL model
def get_hard_triplet_data(imgs, labels, batch_size, base_network):
    image_a = np.zeros((batch_size, imgs.shape[1], imgs.shape[2],
imgs.shape[3]))
    image_b = np.zeros((batch_size, imgs.shape[1], imgs.shape[2],
imgs.shape[3]))
    image_c = np.zeros((batch_size, imgs.shape[1], imgs.shape[2],
imgs.shape[3]))

    # Precompute embeddings for all images
    embeddings = base_network.predict(imgs)

    for i in range(batch_size):
        # Select anchor and positive (same label)
        idx1 = np.random.randint(0, len(imgs))
        idx2 = np.random.randint(0, len(imgs))
        while labels[idx1] != labels[idx2]:
            idx2 = np.random.randint(0, len(imgs))

        # Select hard negative (different label, within a distance range)
        anchor_emb = embeddings[idx1]
        negative_indices = np.where(labels != labels[idx1])[0]
        negative_embs = embeddings[negative_indices]
        distances = np.linalg.norm(negative_embs - anchor_emb, axis=1)
        # Select negatives with distance between 0.3 and 1.5
        valid_negatives = negative_indices[(distances > 0.3) & (distances <
1.5)]

        if len(valid_negatives) == 0:
            # Fallback: select the closest negative if no valid ones are
found
            idx3 = negative_indices[np.argmin(distances)]
        else:
            idx3 = valid_negatives[np.argmin(distances[(distances > 0.3) &
(distances < 1.5)])]
            image_a[i, :, :, :] = imgs[idx1, :, :, :]
            image_b[i, :, :, :] = imgs[idx2, :, :, :]
            image_c[i, :, :, :] = imgs[idx3, :, :, :]

    return [image_a, image_b, image_c]

#
# Data loading and Pre-processing
#

# Load the data
# Change the path to whatever is appropriate on your system
train_X, train_Y, val_X, val_Y, gallery_X, gallery_Y, probe_X, probe_Y =
load_data('/content/drive/MyDrive/CAB420/ASSESSMENTS/ASSESSMENT_1B/Q1')

# Pre-processing

```

```
# Do any resizing, colour conversion, etc, here. You can use the resize and
convert_to_grayscale functions defined above. If
# you're keen, you could also look at other colour spaces - however this is
not needed
#
# Note that you will do the same pre-processing to all datasets
train_X_resized = resize(train_X, (64, 32))
val_X_resized = resize(val_X, (64, 32))
gallery_X_resized = resize(gallery_X, (64, 32))
probe_X_resized = resize(probe_X, (64, 32))

# get validation gallery and probe sets, do this after the pre-processing to
avoid double handling the data
val_gallery_X, val_gallery_Y, val_probe_X, val_probe_Y =
sample_gallery_and_probe(val_X_resized, val_Y)
val_gallery_X_resized = val_gallery_X
val_probe_X_resized = val_probe_X

# Create a vectorised version of the data
# You will need this to develop your non-deep learning method. We recommend
that you do your vectorisation after you've
# done any pre-processing such that you are using the exact same data for
both the non-DL and DL systems.
#
# Note that the code below will vectorise the raw version of the data. If
you have done any resizing, colour conversion, etc,
# change this to vectorise that version of the data
train_X_vec = vectorise(train_X_resized)
val_X_vec = vectorise(val_X_resized)
val_gallery_X_vec = vectorise(val_gallery_X_resized)
val_probe_X_vec = vectorise(val_probe_X_resized)
gallery_X_vec = vectorise(gallery_X_resized)
probe_X_vec = vectorise(probe_X_resized)

#
# Non-DL Method
#
# Using the vectorised data, develop your non-DL method. A suggested approach
would be:
# - Train a PCA, LDA, or both PDA and LDA subspace using the training data.
Note that you will need to select the method(s),
# and choose any key parameters (for example, if you use PCA how many
components you retain)
# - Using the trained subspace, project the gallery and probe data.
```

```
#
# If you wish to use the validation set:
# - Use the val_gallery and val_probe sets to determine the transform to
#   use / number of components to keep. You would compute
#   CMC curves / rank-1 accuracy for different options and pick the one
#   that works best on the validation set (see below for
#   info on computing CMC curves, etc).
#

# Hyperparameter tuning for PCA components
n_components_list = [50, 100, 200, 300, 500]
best_n_components = 100
best_top1_acc = 0

for n_components in n_components_list:
    if n_components < len(train_X_vec) - len(np.unique(train_Y)):
        top1_acc = evaluate_pca_lda(n_components, train_X_vec, train_Y,
                                    val_gallery_X_vec, val_gallery_Y,
                                    val_probe_X_vec, val_probe_Y)

        if top1_acc > best_top1_acc:
            best_top1_acc = top1_acc
            best_n_components = n_components

# Train final PCA + LDA model
print("N component", best_n_components)
print("best top 1", best_top1_acc)
pca = PCA(n_components=best_n_components)
pca.fit(train_X_vec)
train_pca = pca.transform(train_X_vec)
gallery_pca = pca.transform(gallery_X_vec)
probe_pca = pca.transform(probe_X_vec)

lda = LinearDiscriminantAnalysis()
lda.fit(train_pca, train_Y)
NONDLgallery_feats = lda.transform(gallery_pca)
NONDLprobe_feats = lda.transform(probe_pca)

# Evaluate non-deep learning method
NONDLranked_hist = get_ranked_histogram_l2_distance(NONDLgallery_feats,
gallery_Y, NONDLprobe_feats, probe_Y)
NONDLcmc = ranked_hist_to_CMC(NONDLranked_hist)
NONDLrank_1_acc = NONDLcmc[0]
NONDLrank_5_acc = NONDLcmc[4]
NONDLrank_10_acc = NONDLcmc[9]
NONDLrank_50_acc = NONDLcmc[49]
NONDLrank_100_acc = NONDLcmc[99]
NONDLrank_150_acc = NONDLcmc[149]
print("ranked hist", NONDLranked_hist)
print("rank 1", NONDLrank_1_acc)
print("rank 5", NONDLrank_5_acc)
print("rank 10", NONDLrank_10_acc)
print("rank 50", NONDLrank_50_acc)
print("rank 100", NONDLrank_100_acc)
print("rank 150", NONDLrank_150_acc)
print("cmc", NONDLcmc)
plot_cmc(NONDLcmc)

#
# DL Method
#
```

```
# Using the image data, develop your DL method. A suggested approach would
be:
# - Pick a backbone for your network. This could be a network from the
lecture or prac examples, or something from
#   keras.applications. As per Assignment 1A, the network does not need to
be overly complex, but you should briefly explain
#   your selection of this model. You should avoid networks so simple that
they perform very badly.
# - Pick your training approach. You have been given pair and triplet
creation functions above, so you can easily use either contrastive
#   or a triplet loss. You could also look at the bonus example on hard
negative mining and consider this approach. Once you've
#   picked your approach, set the network up with the correct number of
inputs and an appropriate loss.
# - Create some pairs and triplets, and train the network using the training
set
#
# If you wish to use the validation set:
# - Use the val_X and val_Y set with your pair/triplet/whatever function to
determine when your model has converged
# - Use the val_gallery and val_probe sets to determine other network
parameters such as embedding sizes. You would compute
#   CMC curves / rank-1 accuracy for different options and pick the one
that works best on the validation set (see below for
#   info on computing CMC curves, etc).
#

# Fixed hyperparameters
embedding_size = 256 # Reduced from 512
batch_size = 64
epochs = 50
margin = 0.5

# Train initial model for hard negative mining
input_shape = (64, 32, 3)
dummy_input = keras.Input(input_shape)
base_network = vgg_net(dummy_input, [24, 48, 96], [128],
spatial_dropout=0.2, dropout=0.2)
fc_output = base_network
fc_output = layers.BatchNormalization()(fc_output) # Add BatchNormalization
before embedding
embedding_layer = layers.Dense(embedding_size, activation=None)(fc_output)
embedding_layer = layers.Lambda(lambda x: tf.math.l2_normalize(x,
axis=1))(embedding_layer)
embedding_layer = layers.Dropout(0.3)(embedding_layer)
base_network = keras.Model(dummy_input, embedding_layer,
name='TripletBranch')

input_anchor = keras.Input(input_shape, name='Anchor')
input_positive = keras.Input(input_shape, name='Positive')
input_negative = keras.Input(input_shape, name='Negative')

embedding_anchor = base_network(input_anchor)
embedding_positive = base_network(input_positive)
embedding_negative = base_network(input_negative)

loss_layer = TripletLossLayer(alpha=margin,
name='triplet_loss_layer')([embedding_anchor, embedding_positive,
embedding_negative])
```

```

triplet_network = keras.Model(inputs=[input_anchor, input_positive,
input_negative], outputs=loss_layer)
triplet_network.compile(optimizer=keras.optimizers.RMSprop(learning_rate=0.
0001, clipnorm=1.0))

# Initial triplets for precomputing embeddings
triplet_train_x = get_triplet_data(train_X_resized, train_Y, 20000)
triplet_val_x = get_triplet_data(val_X_resized, val_Y, 1000)

# Train with hard negative mining
triplet_train_x = get_hard_triplet_data(train_X_resized, train_Y, 20000,
base_network)
triplet_val_x = get_hard_triplet_data(val_X_resized, val_Y, 1000,
base_network)

triplet_network.fit(triplet_train_x, batch_size=batch_size, epochs=epochs,
validation_data=(triplet_val_x, None))

# Evaluate deep learning method
DLgallery_feats = base_network.predict(gallery_X_resized)
DLprobe_feats = base_network.predict(probe_X_resized)
DLranked_hist = get_ranked_histogram_l2_distance(DLgallery_feats,
gallery_Y, DLprobe_feats, probe_Y)
DLcmc = ranked_hist_to_CMC(DLranked_hist)
DLrank_1_acc = DLcmc[0]
DLrank_5_acc = DLcmc[4]
DLrank_10_acc = DLcmc[9]
DLrank_50_acc = DLcmc[49]
DLrank_100_acc = DLcmc[99]
DLrank_150_acc = DLcmc[149]
print("ranked hist", DLranked_hist)
print("rank 1", DLrank_1_acc)
print("rank 5", DLrank_5_acc)
print("rank 10", DLrank_10_acc)
print("rank 50", DLrank_50_acc)
print("rank 100", DLrank_100_acc)
print("rank 150", DLrank_150_acc)
print("cmc", DLcmc)
plot_cmc(DLcmc)

#
# Evaluation
#
# You will now evaluate your methods using the CMC curve and rank-1, -5 and
-10 accuracies. The functions provided above will allow
# you to obtain these values. To do this you will use code that is something
like the following:
#
# Comparison: Plot CMC curves together
fig = plt.figure(figsize=[10, 8])
ax = fig.add_subplot(1, 1, 1)
ax.plot(range(1, len(NONDLcmc)+1), NONDLcmc, label='Non-DL (PCA+LDA)')
ax.plot(range(1, len(DLcmc)+1), DLcmc, label='DL (Triplet)')
ax.set_xlabel('Rank')
ax.set_ylabel('Count')
ax.set_ylim([0, 1.0])
ax.set_title('CMC Curve Comparison')
ax.legend()

# Print accuracies for comparison
print("Non-Deep Learning (PCA + LDA) Accuracies:")

```

```
print(f"Top-1: {NONDLrank_1_acc:.4f}")
print(f"Top-5: {NONDLrank_5_acc:.4f}")
print(f"Top-10: {NONDLrank_10_acc:.4f}")
print(f"Top-50: {NONDLrank_50_acc:.4f}")
print(f"Top-100: {NONDLrank_100_acc:.4f}")
print(f"Top-150: {NONDLrank_150_acc:.4f}")
print("\nDeep Learning (Triplet Network) Accuracies:")
print(f"Top-1: {DLrank_1_acc:.4f}")
print(f"Top-5: {DLrank_5_acc:.4f}")
print(f"Top-10: {DLrank_10_acc:.4f}")
print(f"Top-50: {DLrank_50_acc:.4f}")
print(f"Top-100: {DLrank_100_acc:.4f}")
print(f"Top-150: {DLrank_150_acc:.4f}")
#
# In the above, the model.transform operation will be replaced with whatever
# is appropriate for what you have done. For the non-DL
# method, this is likely to be fairly similar. For the DL method, you will
# likely change this to call the predict function on your
# backbone network. Your input data (gallery_X and probe_X) will also change
# to any resized, colour converted and/or vectorised
# version you are using.
#
# The cmc variable that results from the call to ranked_hist_to_CMC will
# give you the rank-1, -5 and -10 accuracies directly, and
# is what you will plot to visualise performance. Note that you may wish to
# plot the CMCs for both models on the same axis to make
# it easier to compare performance.
#
```

Appendix 3B: Code for Problem 2

```
# 1. Imports
!pip install --upgrade tensorflow_datasets
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'
import matplotlib.pyplot as plt
from matplotlib import gridspec
import tensorflow as tf
import tensorflow_datasets as tfds
import keras
from keras.layers import Input, Dense, Conv2D, MaxPooling2D, UpSampling2D,
concatenate, BatchNormalization, SpatialDropout2D, Activation, Flatten,
GlobalAveragePooling2D
from keras.models import Model
import numpy as np
import pandas as pd
import glob
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
# Clear the session at the beginning to start with a clean slate
tf.keras.backend.clear_session()

# 2. Def Functions
# Template Functions (Unmodified)
def preprocess_segmentation_mask(segmentation_mask):
    """preprocess the segmentation mask

    The original segmentation mask has three categories.
        foreground, background and outline
    This function will just convert it to foreground and background

    The original segmentation mask is also 1-index, so will convert it
    to 0-index.

    the original mask is represented as:
    1 - edge of dog/cat and things like leashes etc.
    2 - background
    3 - foreground

    we want to just merge the edges and foreground of the doggo/catto, and
    then treat it as a binary semantic segmentation task.
    To achieve this, we will just subtract two, converting to values of [-
1, 0, 1],
    and then apply the abs function to convert the -1 values (edges) to the
    foreground.

    Will also convert it to 32 bit float which will be needed for working
    with tf.

    Why am I doing it this way?
    A reasonable question. Initially I tried to do it with just normal array
    indexing,
    but this is a bit more work since the mask is a tensorflow tensor and
    not a np array.
    We could alternatively convert it to an array, perform indexing and
    then map it back,
    but this would have a performance overhead, which wouldn't be a big
    deal, but still.
    With all that being said, I am doing it for you, so you don't have to.
```

```
Args:
    segmentation_mask (array):
        original segmentation mask

Returns:
    preprocessed segmentation_mask
"""
return tf.abs(tf.cast(segmentation_mask, tf.float32) - 2)

def return_image_label_mask(ds_out):
    """ function to return image, class label and segmentation mask

    The original dataset contains additional information, such as the
    filename and
    the species. We don't care about any of that for this work, so will
    discard them and just keep the original image as our input, and then
    a tuple of our outputs that will be the class label and the semantic
    segmentation mask.

    Whilst we are here, we will also preprocess the segmentation mask.

    Args:
        ds_out: dict
            original dataset output

    Returns:
        RGB image
        tuple of class label and preprocessed segmentation mask
    """
    # preprocess the segmentation mask
    seg_mask = preprocess_segmentation_mask(ds_out['segmentation_mask'])
    image = tf.cast(ds_out['image'], tf.float32)
    # image = standardise_image(image)
    return image, (ds_out['label'], seg_mask)

def mobilenet_preprocess_image(image):
    """Apply preprocessing that is suitable for MobileNetV3.

    Simply scales to ranges [-1, 1]

    you should use this preprocessing for both your model and the mobilenet
    model
    """
    image = (image - 127.5) / 255.0
    return image

def unprocess_image(image):
    """ undo preprocessing above so can plot images"""
    image = image * 255.0 + 127.5
    return image

def preprocess_and_resize(image, output, image_size):
    """apply preprocessing steps above to images and resize images and maps

    Each image in the dataset is of a different size. The resizing will make
    sure
    each image is the same size.
    """
    # resize the image and the semantic segmentation mask
    image = tf.image.resize(image, [image_size, image_size])
    image = mobilenet_preprocess_image(image)
```

```

mask = tf.image.resize(output[1], [image_size, image_size])
return image, (output[0], mask)

def flip_lr_augmentation(image, output, flip_lr_prob):
    """ function to return perform left-right flip augmentation

    The function will flip the image along the left-right axis with
    a defined probability.
    """

    # randomly sample a value between 0 and 1
    uniform_sample = tf.random.uniform([], minval=0, maxval=1)
    # perform flip_lr with probability given by flip_lr_prob
    flip_lr_cond = tf.math.less(uniform_sample, flip_lr_prob)
    # output is a tuple of (class, segmentation_mask), pull out the
    segmentation mask
    seg = output[1]

    # wrapper fn for when we do the flip
    def flip():
        flipped_image = tf.image.flip_left_right(image)
        flipped_seg = tf.image.flip_left_right(seg)
        return flipped_image, flipped_seg

    # wrapper fn for when we do NOT flip
    def no_flip():
        return image, seg

    # apply augmentation
    image, seg = tf.cond(flip_lr_cond, flip, no_flip)
    # return the image, and output
    return image, (output[0], seg)

def select_tasks(image, output, classification=True, segmentation=True):
    """select the tasks to include the data

    By default for each input there are two outputs. This function allows
    you to select which outputs to use, so the problem can be reduced to a
    single task problem for initial experimenting.
    """
    # both tasks
    if classification and segmentation:
        return image, output
    # just classification
    elif classification:
        return image, output[0]
    # just segmentation
    elif segmentation:
        return image, output[1]
    # neither task, doesn't really make sense, so return the image
    # for a self-supervised task
    else:
        return image, image

class TrainForTime(keras.callbacks.Callback):
    """callback to terminate training after a time limit is reached

    Can be used to control how long training runs for, and will terminate
    training once a specified time limit is reached.
    """
    def __init__(

```

```

        self,
        train_time_mins=15,
    ):
        super().__init__()

        self.train_time_mins = train_time_mins
        self.epochs = 0
        self.train_time = 0
        self.end_early = False

    def on_train_begin(self, logs=None):
        # save the start time
        self.start_time = tf.timestamp()

    def on_epoch_end(self, epoch, logs=None):
        self.epochs += 1
        current_time = tf.timestamp()
        training_time = (current_time - self.start_time)
        if (training_time / 60) > self.train_time_mins:
            self.train_time = current_time - self.start_time
            self.model.stop_training = True
            self.end_early = True

    def on_train_end(self, logs=None):
        if self.end_early:
            print('training time exceeded and ending early')
            print(f'training ended on epoch {self.epochs}')
            print(f'training time = {self.train_time / 60} mins')

def load_oxford_pets(split,
                     batch_size=233,
                     classification=True,
                     segmentation=True,
                     shuffle=True,
                     augment=True,
                     image_size=300):
    """Load Oxford pets dataset for Assignment 1B

    Function handles loading of data for 1b, included processing of images
and
    semantic segmentation masks. This function will
    organise the tensorflow dataset to return an output that is a tuple,
where
    the tuple will be (classification_labels, segmentation_masks).

    Parameters
    -----
    split : string
        either train, val or test string
    classification : bool
        whether to include classification labels
    segmentation : bool
        whether to include semantic segmentation masks
    batch_size : int
        size of batches to use
    shuffle : bool
        whether to shuffle the dataset (WILL ONLY APPLY TO TRAIN)
    augment : bool
        whether to augment the dataset (WILL ONLY APPLY TO TRAIN)
    image_size : int
        new image size

```

```

Returns
-----
    tf.Dataset containing the Oxford pets dataset
"""
# lets do some error checking first
# Check for a valid dataset split, this must be train or test
if (split != 'train') and (split != 'val') and (split != 'test'):
    raise ValueError('Arg for split must be either \'train\' or \'test\'')
if (not classification) and (not segmentation):
    print("WARNING: One of the tasks (classification and segmentation)
must be selected")
    print("Setting both to enabled")
    classification = True
    segmentation = True

    # check that if using the val split, shuffle is false. If not, print a
warning and force shuffle to be false
    if (split == 'val') and shuffle:
        print("WARNING: shuffle is set to true, but have specified split to
be \'val\'")
        print('The shuffle argument will be ignored')
        shuffle = False

    # check that if using the test split, shuffle is false. If not, print a
warning and force shuffle to be false
    if (split == 'test') and shuffle:
        print("WARNING: shuffle is set to true, but have specified split to
be \'test\'")
        print('The shuffle argument will be ignored')
        shuffle = False

    # check that if using the val split, augment is false. If not, print a
warning and force augment to be false
    if (split == 'val') and augment:
        print("WARNING: augment is set to true, but have specified split to
be \'val\'")
        print('The augment argument will be ignored')
        augment = False

    # check that if using the test split, augment is false. If not, print a
warning and force augment to be false
    if (split == 'test') and augment:
        print("WARNING: augment is set to true, but have specified split to
be \'test\'")
        print('The augment argument will be ignored')
        augment = False

    # the dataset by default only has train and test splits. If val is
requested, pull the first 30% of the test set
    if (split == 'val'):
        split = 'test[:30%]'
    # the test set then becomes the remaining 70% of the original test set
    elif (split == 'test'):
        split = 'test[30%:]'

    # now start loading the dataset
    ds = tfds.load('oxford_iiit_pet',
                    split=split,
                    with_info=False)

```

```

# remove unnecessary dataset info
ds = ds.map(return_image_label_mask)

# augmentation
# only apply if in the training split and augment has been set to True
if split == 'train' and augment:
    # apply a left-right flip with 50% probability
    flip_lr_prob = 0.5
    # flip operation
    ds = ds.map(lambda inp, out: flip_lr_augmentation(inp, out,
flip_lr_prob), num_parallel_calls=tf.data.AUTOTUNE)

    # more augmentation operations could go here .....

# Final processing of the data
# here we will resize the data, and add the preprocessing that is needed
for compatability with the mobilenet models.
ds = ds.map(lambda inp, out: preprocess_and_resize(inp, out, image_size))

# and now remove any tasks that we don't want. Note that we call this
last as it means that all the other functions
# can safely assume that data for both tasks is in the dataset
ds = ds.map(lambda inp, out: select_tasks(inp, out, classification,
segmentation))

# if in the training split, and shuffle is true, shuffle the data
if split == 'train' and shuffle:
    ds = ds.shuffle(1000)

# return the loaded and processed dataset
return ds.batch(batch_size).prefetch(tf.data.AUTOTUNE)

# Weekly Example Functions (Unmodified Logic, Adjusted for Import)
def conv_block(inputs, filters, spatial_dropout=0.0, max_pool=True):
    x = keras.layers.Conv2D(filters=filters, kernel_size=(3,3),
padding='same', activation='relu')(inputs)
    x = keras.layers.Conv2D(filters=filters, kernel_size=(3,3),
padding='same', activation=None)(x)
    x = keras.layers.BatchNormalization()(x)
    x = keras.layers.Activation('relu')(x)
    if (spatial_dropout > 0.0):
        x = keras.layers.SpatialDropout2D(spatial_dropout)(x)
    if (max_pool == True):
        x = keras.layers.MaxPool2D(pool_size=(2, 2))(x)
    return x

def fc_block(inputs, size, dropout):
    x = keras.layers.Dense(size, activation=None)(inputs)
    x = keras.layers.BatchNormalization()(x)
    x = keras.layers.Activation('relu')(x)
    if (dropout > 0.0):
        x = keras.layers.Dropout(dropout)(x)
    return x

def vgg_net(inputs, filters, fc, spatial_dropout=0.0, dropout=0.0):
    x = inputs
    for idx, i in enumerate(filters):
        x = conv_block(x, i, spatial_dropout, not (idx==len(filters) - 1))
    x = keras.layers.Flatten()(x)
    for i in fc:
        x = fc_block(x, i, dropout)

```

```

    return x

# 3. Data Loading
image_size = 128 # Reduced to lower memory usage
batch_size = 64  # Reduced to manage compute resources
train_class_seg = load_oxford_pets('train', classification=True,
segmentation=True, shuffle=True, augment=True, batch_size=batch_size,
image_size=image_size)
val_class_seg = load_oxford_pets('val', classification=True,
segmentation=True, shuffle=False, augment=False, batch_size=batch_size,
image_size=image_size)
test_class_seg = load_oxford_pets('test', classification=True,
segmentation=True, shuffle=False, augment=False, batch_size=batch_size,
image_size=image_size)
train_class_seg_ns = load_oxford_pets('train', classification=True,
segmentation=True, shuffle=False, augment=True, batch_size=batch_size,
image_size=image_size)

# Plot a few samples to verify data loading
fig, axs = plt.subplots(2, 3, figsize=(10, 4), layout="constrained")
num_plot = 3
i = 0
for image, output in train_class_seg.take(1).as_numpy_iterator():
    for i in range(num_plot):
        im = axs[0, i].imshow(np.squeeze(unprocess_image(image[i, ...])) /
255.0)
        axs[0, i].set_title(output[0][i])
        axs[0, i].axis('off')
        im = axs[1, i].imshow(np.squeeze(output[1][i, ...]))
        axs[1, i].axis('off')
        i += 1
    if i >= num_plot:
        break
plt.savefig('doggos_cattos.png')

#
# From-scratch DCNN
#
# Put your from-scratch DCNN here. This network will have one input (the
input image), and two outputs (the class, and the segmentation map). You can
# think of your model as having three components:
# - A backbone, which is going to take your input and encode that. This
will possibly be collection of (probably mostly) 2D convolution layers
# and max pooling layers (and for which you may be able to find something
fairly fit for purpose in lecture or prac examples).
# - A classification head, which will take the backbone output and via one
or more dense layers do the classification task. This will likely only
# be a few layers at most.
# - A decoder head, which will do the semantic segmentation. This will look
a lot like the back half of an autoencoder.
#
# As usual, the network does not need to be overly complex, but you will
need to briefly explain your selection of this model, and you should avoid
# networks so simple that they perform very badly.
#
# Take note of the order that the outputs appear in the data loader. The
data loader will give you (classification_output, segmentation_output), so
# your network should have the outputs in the same order, and your losses
should also be in the same order. With your losses, note that:

```

```
# * For the classification task ground truth data is being given as class
labels, so you should use Sparse Categorical Cross Entropy.
# * For the segmentation loss, there are a few options you could use, but a
good starting point is simply binary cross entropy, essentially
# every pixel is a binary classification task (foreground or background)
# * The order of your losses is critical. The dataset object will give you
(classification_targets, segmentation_targets). This means that you're
# network should it's outputs, and loss functions, in the same order - if
you get this order wrong you will get weird errors
#
# When first developing your model, you may want to start by getting a single
output of the network going, and then adding the second output. For
# example, you may build a network with the image input and the just the
classification head, get that running, and then add the decoder head. You
can
# use the classification and segmentation flags in the data loader to turn
off one of the outputs.

# For model training, we have provided a callback, TrainForTime, which will
terminate training after some time limit is reached. You could use this
# as follows:
#
# train_time = 15 # 15-minute training limit
# my_awesome_model.fit(train_class_seg, epochs=epochs, verbose=True,
validation_data=val_class_seg, callbacks=[TrainForTime(train_time)])
#
# You don't have to use this, but you may wish to use this to ensure training
doesn't take too long.
# Note that if you don't train until convergence, you will need to explain
what you observe with training and what the implications of this are.

# Build the from-scratch model with a shared backbone, classification head,
and segmentation head

train_time = 15

# 4. From-Scratch DCNN
# Build the from-scratch model with a shared backbone, classification head,
and segmentation head
input_shape = (image_size, image_size, 3)
inputs = Input(shape=input_shape)
conv1 = conv_block(inputs, 32, spatial_dropout=0.2, max_pool=True) # (64,
64, 32)
conv2 = conv_block(conv1, 64, spatial_dropout=0.2, max_pool=True) # (32,
32, 64)
conv3 = conv_block(conv2, 128, spatial_dropout=0.2, max_pool=True) # (16,
16, 128)
class_head = GlobalAveragePooling2D()(conv3) # (128,)
class_output = Dense(37, activation='softmax',
name='class_output')(class_head)
seg_head = UpSampling2D((2, 2))(conv3) # (32, 32, 128)
seg_head = concatenate([seg_head, conv2]) # (32, 32, 192)
seg_head = conv_block(seg_head, 64, spatial_dropout=0.2, max_pool=False) #
(32, 32, 64)
seg_head = UpSampling2D((2, 2))(seg_head) # (64, 64, 64)
seg_head = concatenate([seg_head, conv1]) # (64, 64, 96)
seg_head = conv_block(seg_head, 32, spatial_dropout=0.2, max_pool=False) #
(64, 64, 32)
seg_head = UpSampling2D((2, 2))(seg_head) # (128, 128, 32)
seg_head = Conv2D(16, (3, 3), padding='same',
activation='relu')(seg_head) # (128, 128, 16)
```

```

seg_head = BatchNormalization()(seg_head)
seg_output = Conv2D(1, (3, 3), padding='same', activation='sigmoid',
name='seg_output')(seg_head) # (128, 128, 1)
# 15-min
from_scratch_model = Model(inputs=inputs, outputs=[class_output,
seg_output])
from_scratch_model.compile(optimizer=keras.optimizers.Adam(learning_rate=0.
001), loss=['sparse_categorical_crossentropy', 'binary_crossentropy'],
loss_weights=[1.0, 1.0], metrics={'class_output': 'accuracy', 'seg_output':
'accuracy'})
# Train the Model
from_scratch_model.fit(train_class_seg, epochs=200, verbose=True,
validation_data=val_class_seg, callbacks=[TrainForTime(train_time)])

#
# Fine-tuned MobileNet
#
# Put your mobilenet model here. This will have the same structure as the
from-scratch network above, but now the backbone will be mobilenet rather
# than something of your own design. The classification and segmentation
heads will possibly be quite similar to what you have above. As per the
# from-scratch network, you may want to build this with just one output to
start with, and then add the second when you get that working.

# build the rest of the model here

# train the model. As per the from-scratch model, you may wish to use the
callback - though if the model is not trained until convergence you will
# need to explain the implications of this.

# 5. MobileNet
# Load MobileNetV3Small backbone
mobile_base = keras.applications.MobileNetV3Small(input_shape=(image_size,
image_size, 3),
include_top=False,
include_preprocessing=False)

# Programmatically find layers for skip connections by checking output shapes
# We need layers with output shapes approximately (16, 16, channels) and (8,
8, channels) for image_size=128
input_tensor = Input(shape=(image_size, image_size, 3))
shape_dict = {}
for layer in mobile_base.layers:
    output = layer.output
    temp_model = Model(inputs=mobile_base.input, outputs=output)
    output_shape = temp_model.output_shape
    shape_dict[layer.name] = output_shape

# Find layers with spatial dimensions close to 16x16 and 8x8
layer_16x16 = None
layer_8x8 = None
for layer_name, shape in shape_dict.items():
    if len(shape) == 4: # (batch, height, width, channels)
        height = shape[1]
        if height == 16 and layer_16x16 is None:
            layer_16x16 = layer_name
        if height == 8 and layer_8x8 is None:
            layer_8x8 = layer_name

```

```
# Print the selected layers and their shapes for verification
print(f"Selected layer for 16x16: {layer_16x16}, Shape:
{shape_dict[layer_16x16]}")
print(f"Selected layer for 8x8: {layer_8x8}, Shape:
{shape_dict[layer_8x8]}")

# Extract intermediate layers for skip connections
conv_16x16 = mobile_base.get_layer(layer_16x16).output
conv_8x8 = mobile_base.get_layer(layer_8x8).output

# Classification Head
class_head = GlobalAveragePooling2D()(mobile_base.output) # (576,)
class_output = Dense(37, activation='softmax',
name='class_output')(class_head)

# Segmentation Head (with Skip Connections)
seg_head = UpSampling2D((2, 2))(conv_8x8) # (16, 16, channels)
seg_head = concatenate([seg_head, conv_16x16]) # Concatenate with 16x16
layer
seg_head = conv_block(seg_head, 64, spatial_dropout=0.2, max_pool=False) #
(16, 16, 64)
seg_head = UpSampling2D((2, 2))(seg_head) # (32, 32, 64)
seg_head = conv_block(seg_head, 32, spatial_dropout=0.2, max_pool=False) #
(32, 32, 32)
seg_head = UpSampling2D((2, 2))(seg_head) # (64, 64, 32)
seg_head = conv_block(seg_head, 16, spatial_dropout=0.2, max_pool=False) #
(64, 64, 16)
seg_head = UpSampling2D((2, 2))(seg_head) # (128, 128, 16)
seg_output = Conv2D(1, (3, 3), padding='same', activation='sigmoid',
name='seg_output')(seg_head) # (128, 128, 1)

# Model
mobilenet_model = Model(inputs=mobile_base.input, outputs=[class_output,
seg_output])

# Training: Stage 1 (Freeze backbone, train heads)
mobile_base.trainable = False
mobilenet_model.compile(optimizer=keras.optimizers.Adam(learning_rate=0.001
), loss=['sparse_categorical_crossentropy', 'binary_crossentropy'],
loss_weights=[1.0, 1.0], metrics={'class_output': 'accuracy', 'seg_output':
'accuracy'})
mobilenet_model.fit(train_class_seg, epochs=10, verbose=True,
validation_data=val_class_seg, callbacks=[TrainForTime(train_time)])

# Training: Stage 2 (Unfreeze last 3 blocks, fine-tune)
# Identify layers to unfreeze (e.g., last 3 blocks)
for layer in mobile_base.layers[-30:]: # Approximate last 3 blocks
    layer.trainable = True
mobilenet_model.compile(optimizer=keras.optimizers.Adam(learning_rate=1e-
5), loss=['sparse_categorical_crossentropy', 'binary_crossentropy'],
loss_weights=[1.0, 1.0], metrics={'class_output': 'accuracy', 'seg_output':
'accuracy'})
mobilenet_model.fit(train_class_seg, epochs=200, verbose=True,
validation_data=val_class_seg, callbacks=[TrainForTime(train_time)])

#
# Evaluation
#
```

```

# You should evaluate both your networks for both tasks. To help generate
metrics and plots, you may want to pull out the labels from the dataset
# which you could do with something like the following:
#
#     y_class_test      =      np.concatenate([y[0]      for      x,y      in
test_class_seg.as_numpy_iterator()])
#     y_seg_test       =      np.concatenate([y[1]      for      x,y      in
test_class_seg.as_numpy_iterator()])
#
# For evaluating the two tasks, the classification task is just like all the
other classification tasks you've done this semester and you can follow
# how these have been dealt with. For the segmentation task, one way to look
at this is as a binary classification task as you're classifying each
# pixel into foreground and background. With that in mind you could do
something like this:
#
# class_preds, seg_preds = my_awesome_network.predict(test_class_seg)
# seg_preds_flat = seg_preds.reshape(-1) > 0.5 # flatten predictions, turn
into boolean vector where true is fgnd and false is bkgnd
# seg_gt_flat = y_seg_test.reshape(-1) > 0.5 # same as above, but for the
ground truth, need to have both as a boolean vector
# cm = sklearn.metrics.confusion_matrix(seg_gt_flat, seg_preds_flat)
#
# You could also look at other segmentation specific metrics as a way to
evaluate performance

# 6. Evaluation
# Process ground truth and predictions in chunks to reduce memory usage
def evaluate_model(model, dataset, num_samples=100):
    class_preds_list = []
    seg_preds_list = []
    y_class_list = []
    y_seg_list = []

    for batch in dataset.take(num_samples // batch_size + 1):
        images, (labels, masks) = batch
        class_pred, seg_pred = model.predict(images, verbose=0)
        class_preds_list.append(class_pred)
        seg_preds_list.append(seg_pred)
        y_class_list.append(labels.numpy())
        y_seg_list.append(masks.numpy())

    class_preds = np.concatenate(class_preds_list, axis=0)[:num_samples]
    seg_preds = np.concatenate(seg_preds_list, axis=0)[:num_samples]
    y_class = np.concatenate(y_class_list, axis=0)[:num_samples]
    y_seg = np.concatenate(y_seg_list, axis=0)[:num_samples]

    return class_preds, seg_preds, y_class, y_seg

# From-Scratch Model Evaluation (Limit to 100 samples to reduce memory usage)
class_preds_scratch, seg_preds_scratch, y_class_test, y_seg_test =
evaluate_model(from_scratch_model, test_class_seg, num_samples=100)

# Classification: Accuracy and Confusion Matrix
class_preds_scratch_labels = np.argmax(class_preds_scratch, axis=1)
cm_scratch = confusion_matrix(y_class_test, class_preds_scratch_labels)
fig = plt.figure(figsize=[10, 10])
ax = fig.add_subplot(1, 1, 1)
ConfusionMatrixDisplay(cm_scratch).plot(ax=ax)
ax.set_title('From-Scratch Model: Classification Confusion Matrix')
plt.savefig('from_scratch_class_confusion.png')

```

```

class_accuracy_scratch = np.mean(class_preds_scratch_labels == y_class_test)

# Segmentation: IoU and Confusion Matrix
seg_preds_scratch_flat = (seg_preds_scratch.reshape(-1) > 0.5)
seg_gt_flat = (y_seg_test.reshape(-1) > 0.5)
cm_seg_scratch = confusion_matrix(seg_gt_flat, seg_preds_scratch_flat)
fig = plt.figure(figsize=[10, 10])
ax = fig.add_subplot(1, 1, 1)
ConfusionMatrixDisplay(cm_seg_scratch, display_labels=['Background',
'Foreground']).plot(ax=ax)
ax.set_title('From-Scratch Model: Segmentation Confusion Matrix')
plt.savefig('from_scratch_seg_confusion.png')

# Compute IoU for Segmentation
intersection = np.logical_and(seg_preds_scratch_flat, seg_gt_flat).sum()
union = np.logical_or(seg_preds_scratch_flat, seg_gt_flat).sum()
iou_scratch = intersection / union if union > 0 else 0

# MobileNetV3Small Model Evaluation (Limit to 100 samples)
class_preds_mobile, seg_preds_mobile, y_class_test, y_seg_test =
evaluate_model(mobilenet_model, test_class_seg, num_samples=100)

# Classification: Accuracy and Confusion Matrix
class_preds_mobile_labels = np.argmax(class_preds_mobile, axis=1)
cm_mobile = confusion_matrix(y_class_test, class_preds_mobile_labels)
fig = plt.figure(figsize=[10, 10])
ax = fig.add_subplot(1, 1, 1)
ConfusionMatrixDisplay(cm_mobile).plot(ax=ax)
ax.set_title('MobileNetV3Small Model: Classification Confusion Matrix')
plt.savefig('mobilenet_class_confusion.png')

class_accuracy_mobile = np.mean(class_preds_mobile_labels == y_class_test)

# Segmentation: IoU and Confusion Matrix
seg_preds_mobile_flat = (seg_preds_mobile.reshape(-1) > 0.5)
cm_seg_mobile = confusion_matrix(seg_gt_flat, seg_preds_mobile_flat)
fig = plt.figure(figsize=[10, 10])
ax = fig.add_subplot(1, 1, 1)
ConfusionMatrixDisplay(cm_seg_mobile, display_labels=['Background',
'Foreground']).plot(ax=ax)
ax.set_title('MobileNetV3Small Model: Segmentation Confusion Matrix')
plt.savefig('mobilenet_seg_confusion.png')

intersection = np.logical_and(seg_preds_mobile_flat, seg_gt_flat).sum()
union = np.logical_or(seg_preds_mobile_flat, seg_gt_flat).sum()
iou_mobile = intersection / union if union > 0 else 0

# Print Metrics
print("From-Scratch Model Metrics:")
print(f"Classification Accuracy: {class_accuracy_scratch:.4f}")
print(f"Segmentation IoU: {iou_scratch:.4f}")
print("\nMobileNetV3Small Model Metrics:")
print(f"Classification Accuracy: {class_accuracy_mobile:.4f}")
print(f"Segmentation IoU: {iou_mobile:.4f}")

# Visualize Segmentation Predictions (5 Samples)
fig, axs = plt.subplots(3, 5, figsize=(15, 9), layout="constrained")
for i in range(5):
    # Ground Truth Image
    batch = test_class_seg.take(1).as_numpy_iterator().next()

```

```
axs[0, i].imshow(np.squeeze(unprocess_image(batch[0][i, ...])) / 255.0)
axs[0, i].set_title(f'Sample {i+1}: Ground Truth Image')
axs[0, i].axis('off')
# Ground Truth Mask
axs[1, i].imshow(np.squeeze(y_seg_test[i, ...]))
axs[1, i].set_title('Ground Truth Mask')
axs[1, i].axis('off')
# Predicted Mask (MobileNetV3Small)
axs[2, i].imshow(np.squeeze(seg_preds_mobile[i, ...]))
axs[2, i].set_title('MobileNet Predicted Mask')
axs[2, i].axis('off')
plt.savefig('segmentation_predictions.png')
```